# Homework 3

Davide Quartucci
Student ID: 2191105

**Excercise:** The goal of this exercise is to compute the numerical solution of a boundary value problem governed by the Poisson equation using the Finite Element Method (FEM) with linear triangular elements ($P_1$). Let $\Omega \subset \mathbb{R}^2$ be the computational domain, and $\Gamma = \partial\Omega$ its boundary. The boundary is partitioned into two non-overlapping regions: the Dirichlet boundary $\Gamma_D = \Gamma_{D,1} \cup \Gamma_{D,2}$ and the Neumann boundary $\Gamma_N$, such that $\Gamma_D \cap \Gamma_N = \emptyset$.

The strong form of the problem requires finding a scalar field $u(\mathbf{x})$ such that:

$$\begin{cases} -\partial^2_{x^2} u(\mathbf{x}) - \partial^2_{y^2} u(\mathbf{x}) = f(\mathbf{x}) & \text{in } \Omega \\ u(\mathbf{x}) = d_1(\mathbf{x}) & \text{on } \Gamma_{D,1} \\ u(\mathbf{x}) = d_2(\mathbf{x}) & \text{on } \Gamma_{D,2} \\ \partial_n u(\mathbf{x}) = q(\mathbf{x}) & \text{on } \Gamma_N \end{cases} \tag{1}$$

For the specific problem under analysis, the source term and boundary data are defined as follows:

- Source term: $f(\mathbf{x}) = 0$ everywhere in $\Omega$ (making the equation homogeneous, i.e., Laplace equation).
- Dirichlet Conditions:
  - On $\Gamma_{D,1}$ (top-left edge): $d_1(\mathbf{x}) = 0$.
  - On $\Gamma_{D,2}$ (top-right edge): $d_2(\mathbf{x}) = 1$.
- Neumann Condition: On the remaining boundary $\Gamma_N$ (internal circle and external square borders), the flux is null: $q(\mathbf{x}) = 0$ (Homogeneous Neumann).

## 1 Analysis and Numerical Methodology

The numerical solution is achieved through a modular MATLAB code, structured according to the flowchart presented in **Figure 2** of the assignment. The main script, `main.m`, guides the simulation by looping over the refinement levels ($k = 0 \dots 4$) and calling specific functions for the assembly and boundary condition enforcement tasks as required.

### 1.1 Input Data

For each refinement level, the code first loads the computational grid and the problem data. As defined in **Section 3.1**, the mesh is described by three ASCII files given in the `inputfiles` directory:

- `.coord`: Contains the physical coordinates $(x, y)$ of the mesh vertices.
- `.topol`: Defines the connectivity, listing the three node indices for each triangular element (topology).
- `.bound`: Lists the nodes belonging to the Dirichlet boundary $\Gamma_D$ along with their prescribed values.

- `.dat`: A reference solution file containing the coordinates $(x_{ref}, y_{ref})$ and the solution values $u_{ref}$ computed on a very refined grid. This dataset is used as the ground truth for the error analysis.

Additionally, the code loads a `.trace` file containing the ordered list of nodes along the Neumann boundary $\Gamma_N$, which is used later for the validation of the results (in Requirement Solution section). Once the data is loaded, the script computes the mesh characteristic size $h_{max}$ (the maximum edge length in the triangulation) to track the refinement process.

### 1.2 Global Stiffness Matrix Construction

The assembly process is implemented in the function `assemble_system.m`. Following the algorithm described in **Section 3.2** of the assignment, the code iterates over all triangular elements $k = 1, \ldots, N_{elem}$. For each element, the routine computes the geometric gradients $(\nabla\phi)$ and the local stiffness matrix $H^{(e)}$. These local contributions are then directly accumulated into the global sparse matrix $H$ using the mapping provided by the mesh topology:

$$H_{R,C} \leftarrow H_{R,C} + H^{(e)}_{r,c}$$

This direct update strategy ensures an exact correspondence between the theoretical formulation and the numerical implementation. The global matrix is initialized using MATLAB's `spalloc` function to efficiently handle the sparsity pattern as described in the code.

### 1.3 Nodal Areas Computation

Within the same loop over the elements, the function computes the representative nodal areas required for the $L^2$ error analysis. As requested one-third of each element's area $\Delta_e$ is distributed to its three vertices. This results in a vector $A_{nodal}$ where each entry represents the area weight associated with a specific node.

### 1.4 Boundary Conditions Enforcement

The imposition of boundary conditions is handled by the function `apply_boundary_conditions.m`, distinguishing between natural (Neumann) and essential (Dirichlet) conditions.

**Neumann Conditions:** Since the problem prescribes homogeneous Neumann conditions $(q = 0)$ on $\Gamma_N$, the boundary integral in the weak formulation vanishes naturally. Consequently, no modification of the linear system is required for these nodes.

**Dirichlet Conditions:** For nodes on $\Gamma_D$, the code implements the lifting procedure described in **Algorithm 3.2** to enforce the values $u_i = u_{D,i}$ while preserving the symmetry of the stiffness matrix $H$ (a requirement for the PCG solver). The implementation proceeds in three steps:

1. Symmetry Preservation: First, the known contributions of the Dirichlet nodes are moved to the right-hand side. The global load vector is updated by subtracting the columns corresponding to constrained nodes: $f \leftarrow f - H_{:,i} \cdot u_{D,i}$ (referring to Alg. 3.2, line 2).

2. System Decoupling: The rows and columns associated with Dirichlet nodes are explicitly zeroed out (Alg. 3.2, lines 1 and 4) to decouple them from the rest of the system.

3. Constraint Enforcement: Finally, the identity equation $1 \cdot u_i = u_{D,i}$ is imposed by setting the diagonal entry $H_{ii} = 1$ and the corresponding RHS term $f_i = u_{D,i}$ (Alg. 3.2, lines 3 and 6).

### 1.5 Linear System Solution (PCG Implementation)

The resulting global stiffness matrix $H$ is symmetric and positive definite. Therefore, the linear system $Hu = f$ is solved using the Preconditioned Conjugate Gradient (PCG) method. As specified in the requirements, the solver is configured with a strict tolerance of $10^{-8}$. Two different preconditioning strategies are compared to assess computational efficiency:

1. **Jacobi Preconditioner:** A simple diagonal scaling ($M = \text{diag}(H)$), implemented via MATLAB's vector operations.
2. **Incomplete Cholesky (ICHOL):** A factorization $L \approx \text{chol}(H)$ computed with the `nofill` option to preserve the sparsity pattern of the original matrix.

### 1.6 Error Computation

Since an exact analytical solution for the horseshoe domain is unavailable, the discretization error is evaluated using the high-resolution reference dataset provided with the assignment. The reference file contains the nodal values $(x_{ref}, y_{ref}, u_{ref})$ computed on a grid significantly finer than our Mesh 4. To compare this solution with the coarser meshes used in our simulation ($k = 0 \dots 4$), we adopted the interpolation procedure described in Algorithm 3.3:

1. **Interpolation:** The reference values $u_{ref}$ are mapped onto the nodes of the current mesh using a linear interpolation scheme (implemented via MATLAB's `scatteredInterpolant`).
2. **$L^2$ Norm:** The weighted error norm is computed as:

$$\varepsilon_k = \sqrt{\sum_{i=1}^{N_k} (u_{h,i} - u_{ref}(x_i))^2 \cdot A_i} \tag{2}$$

   where $A_i$ are the nodal areas computed during the assembly phase.

Using this external reference allows us to compute the discretization error even for the finest mesh (Level 4) and verify the convergence rate across the entire refinement range.

## 2 Requirement Solution

In this section, we now present the results obtained by running the FEM code from 0 to 4. The analysis focuses on three main aspects as requested: the qualitative behavior of the solution, the performance of the linear solver (PCG), and the asymptotic convergence rate of the discretization error.

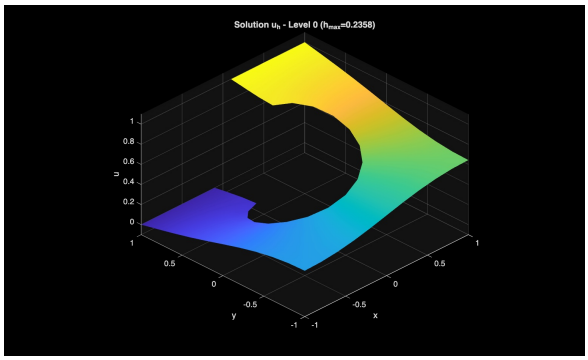
### 2.1 Qualitative Solution

The computed solution $u_h$ for the finest grid is shown in Figure 1. The solution correctly respects the essential boundary conditions:

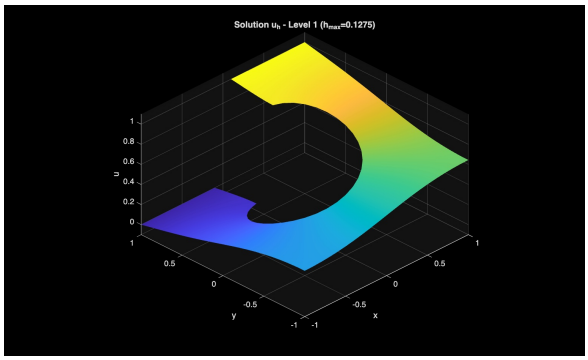
- $u = 0$ on the top-left edge ($\Gamma_{D,1}$).
- $u = 1$ on the top-right edge ($\Gamma_{D,2}$).

Since the source term is null ($f = 0$), the problem reduces to the Laplace equation. The resulting solution represents a steady-state diffusion process (or the equilibrium position of an elastic membrane). As expected, the scalar field $u$ transitions smoothly from the high potential region ($\Gamma_{D,2}$, where $u = 1$) to the low potential region ($\Gamma_{D,1}$, where $u = 0$), following the geometry of the domain without any local extreme inside $\Omega$.

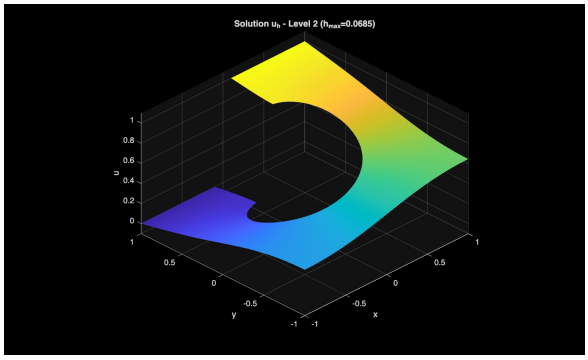
Comparing the subplots, the effect of grid refinement is evident. In the coarser grids (Mesh 0 and 1), the discrete nature of the $P_1$ finite elements is clearly visible, with the solution appearing as a faceted surface composed of planar triangles. As the mesh size $h$ decreases (Mesh 3 and 4), the solution becomes visually indistinguishable from a smooth continuous surface, suggesting that the discretization error has been significantly reduced.



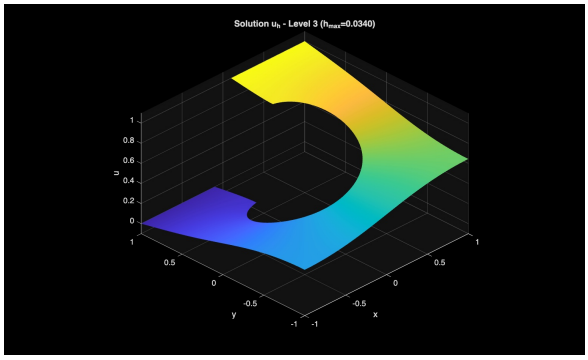

(a) Surface plot of the FEM solution $u_h$ (mesh 0)



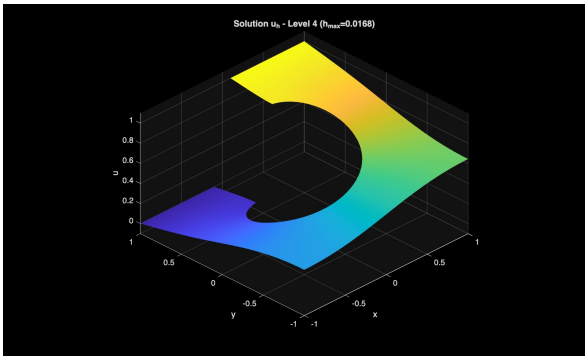

(b) Surface plot of the FEM solution $u_h$ (mesh 1)




(c) Surface plot of the FEM solution $u_h$ (mesh 2)




(d) Surface plot of the FEM solution $u_h$ (mesh 3)



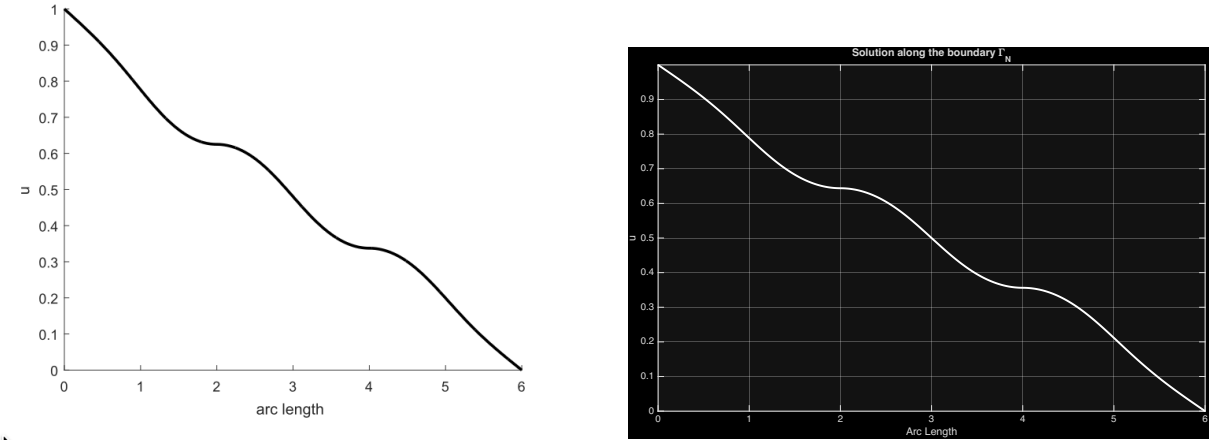

(e) Surface plot of the FEM solution $u_h$ on the finest mesh (mesh 4)

Figure 1: Comparison of plots for meshes from 0 to 4.

## 2.2 Validation against Reference Solution

To validate the correctness of the Finite Element implementation, we extracted the solution values along the Neumann boundary $\Gamma_N$ (the "trace") for the finest mesh (Level 4). From figure 2 the resulting profile exhibits a non-monotonic behavior, starting at $u \approx 1$ and oscillating down to $u \approx 0$, which is consistent with the diffusion process over the horseshoe domain. In addition,

this profile perfectly matches the reference solution provided in **Figure 7** of the assignment text. This qualitative and quantitative agreement confirms that the boundary conditions and the assembly process have been implemented correctly in the FEM code.




(a) Reference Trace (from Assignment) (b) Computed Trace (Mesh 4)

Figure 2: Validation of the numerical solution along the Neumann boundary $\Gamma_N$.

### 2.3 Linear Solver Convergence (PCG Analysis)

We solved the linear system $H\mathbf{u} = \mathbf{f}$ using the Preconditioned Conjugate Gradient method. Figure 5 compares the convergence history (Relative Residual vs. Iterations) for the two preconditioners: Jacobi and Incomplete Cholesky (ICHOL).
**Observations:** as we can see from the semi-log plots:

1. **Impact of Refinement:** as the mesh is refined (from Level 0 to 4), the number of iterations required to reach the tolerance ($10^{-8}$) increases. This is consistent with theory: the condition number of the stiffness matrix scales as $\kappa(H) \approx h^{-2}$, making the system harder to solve for finer meshes every times.

2. **Preconditioner Comparison:**

   - **JACOBI:** exhibits a slower convergence rate. While cheap to apply per iteration, the total iteration count is high, more than double compared to ICHOL on finer meshes.
   - **ICHOL:** shows a much faster convergence (steeper slope in the semi-log plot). The incomplete factorization captures more spectral properties of $H$, drastically reducing the iteration count compared to Jacobi.

### 2.4 FEM Convergence Analysis ($L^2$ Error)

Now to validate the accuracy of the method, we computed the $L^2$ error norm $\varepsilon_k$ for each mesh level $k = 0, \ldots, 4$ with respect to the reference solution given by assignment.

Table 1 summarizes the results. First of all, the experimental convergence ratio $r_k$ used from equation defined before, is reported for each refinement step:

$$r_k = \frac{\varepsilon_{k-1}}{\varepsilon_k} \left( \frac{h_k}{h_{k-1}} \right)^2 \tag{3}$$

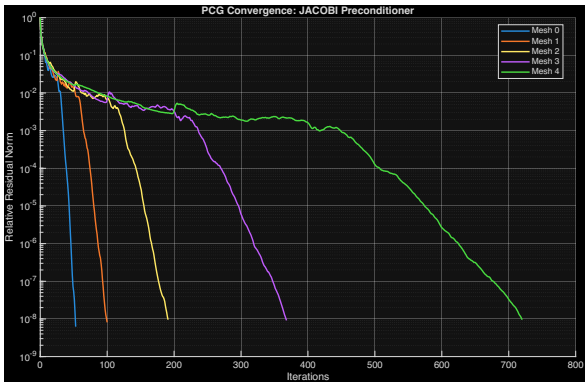



Figure 3: PCG convergence with Jacobi.

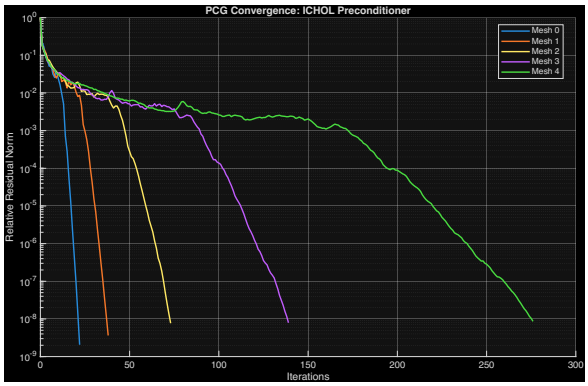



Figure 4: PCG convergence with ICHOL.

Figure 5: Comparison of PCG convergence for different preconditioners across mesh levels.

Ideally, for linear elements ($P_1$) and $L^2$ norm, the error should decrease quadratically ($O(h^2)$). Since the mesh refinement roughly halves the characteristic size $h$ at each step ($h_k \approx h_{k-1}/2$), we expect the error to decrease by a factor of 4. Therefore, if the theoretical rate is met, the ratio $r_k$ should be close to 1.

| **Level** | **Mesh size ($h$)** | **Error ($L^2$)** | **Ratio $r_k$** |
|---|---|---|---|
| 0 | 0.2358 | 2.2872e-03 | - |
| 1 | 0.1275 | 6.0563e-04 | 1.1032 |
| 2 | 0.0685 | 1.4642e-04 | 1.1944 |
| 3 | 0.0340 | 3.6868e-05 | 0.9797 |
| 4 | 0.0168 | 8.7802e-06 | 1.0284 |

Table 1: FEM convergence summary.

**Observations:** The primary goal was to verify the theoretical convergence rate of the FEM. As reported in Table 1, the experimental convergence ratio $r_k$ rapidly converges to the theoretical value of 1. Specifically, the ratio for the finest mesh is $r_4 \approx 1.0284$. This stability indicates that the simulation has successfully entered the asymptotic regime, verifying the expected quadratic convergence rate ($O(h^2)$) as predicted by a priori error estimates for $P_1$ elements.

In addition, the results highlight the inherent trade-off in FEM simulations. While the finest mesh provides the most accurate solution (error $\approx 10^{-5}$), it also leads to a much stiffer linear system. As observed in the PCG analysis, the conditioning of the matrix degrades as $h \to 0$, causing the number of iterations for the simple Jacobi preconditioner to grow significantly. The adoption of the ICHOL preconditioner proved essential to maintain computational efficiency, keeping the iteration count manageable even for the most refined grids.

Finally, the reliability of the implementation was validated by extracting the solution trace along the Neumann boundary $\Gamma_N$. The perfect agreement between the computed profile and the reference data ensures that both the assembly of the stiffness matrix and the enforcement of natural/essential boundary conditions were correctly implemented. □